PROGRAM 14 – ALPHA-BETA PRUNING ALGORITHM

AIM

To write a Python program to implement the Alpha-Beta Pruning algorithm to optimize the Minimax algorithm in gaming.

ALGORITHM

1. Start the program.
2. Create a game tree with terminal values.
3. Initialize Alpha to negative infinity and Beta to positive infinity.
4. Consider MAX and MIN players.
5. MAX selects the maximum value.
6. MIN selects the minimum value.
7. Update Alpha and Beta values during searching.
8. If Alpha becomes greater than or equal to Beta, stop searching that branch.
9. Ignore the unnecessary branches.
10. Display the best value.

CODE

```
def alphabeta(depth,node_index,max_player,values,alpha,beta,max_depth):
    if depth==max_depth:
        return values[node_index]
    if max_player:
        best=float("-inf")
        for i in range(2):
            value=alphabeta(depth+1,node_index*2+i,False,values,alpha,beta,max_depth)
            best=max(best,value)
            alpha=max(alpha,best)
            if alpha>=beta:
                break
        return best
    else:
        best=float("inf")
        for i in range(2):
            value=alphabeta(depth+1,node_index*2+i,True,values,alpha,beta,max_depth)
            best=min(best,value)
```

```
            beta=min(beta,best)
            if alpha>=beta:
                break
        return best
values=[3,5,2,9,12,5,23,23]
depth=3
result=alphabeta(0,0,True,values,float("-inf"),float("inf"),depth)
print("Terminal Values:",values)
print("Best Value using Alpha-Beta Pruning:",result)
```

INPUT

```
values=[3,5,2,9,12,5,23,23]
depth=3
```

OUTPUT

```
Terminal Values: [3, 5, 2, 9, 12, 5, 23, 23]
Best Value using Alpha-Beta Pruning: 12
```

RESULT

Thus, the Alpha-Beta Pruning algorithm was successfully implemented in Python to find the best game value while reducing unnecessary searching.